



1

カードゲーム 2 (Card Game 2)

Author: 萩原 千晴 (Mendako)

小課題 1

$N = 3$ であるので、持っている 3 枚のカードが条件を満たしているかどうかを考える。

すなわち、3 枚のカードに書かれた数値を小さい順に並べ替え、それぞれが $A_1, A_1 + 3, A_1 + 6$ と表せるかどうかを判定すればよい。

```
1    sort(A.begin(), A.end());
2    if(A[0] + 3 == A[1] && A[0] + 6 == A[2]) cout << "Yes" << endl;
3    else cout << "No" << endl;
```

小課題 2

1 から 7 までの整数が書かれているカードから条件を満たすような 3 枚のカードを選ぶには、1, 4, 7 が書かれているカードを選ぶしかない。

したがって、 N 枚のカードの中に 1, 4, 7 が書かれているカードが少なくとも 1 枚以上ずつあるかどうかを判定すればよい。

```
1    bool exist_1 = false, exist_4 = false, exist_7 = false;
2    for(int i = 0; i < N; i++) {
3        if(A[i] == 1) exist_1 = true;
4        if(A[i] == 4) exist_4 = true;
5        if(A[i] == 7) exist_7 = true;
6    }
7    if(exist_1 && exist_4 && exist_7) cout << "Yes" << endl;
8    else cout << "No" << endl;
```



小課題 3

3 枚のカードの組み合わせを $O(N^3)$ で全探索して条件をみたすかどうかを確認し、1 つでも条件をみたす組み合わせがあるかどうかを考えればよい。

```
1      sort(A.begin(), A.end());
2      for(int i = 0; i < N; i++) {
3          for(int j = i + 1; j < N; j++) {
4              for(int k = j + 1; k < N; k++) {
5                  if(A[i] + 3 == A[j] && A[j] + 3 == A[k]) {
6                      cout << "Yes" << endl;
7                      return 0;
8                  }
9              }
10         }
11     }
12     cout << "No" << endl;
```

満点

まず、整数 x ($1 \leq x \leq 200\,000$) が書かれているカードが存在しているかどうかを調べ、情報を配列に保持する。

具体的には、長さ $200\,000$ の配列 $exist$ を「存在していない」という状態で初期化した後、配列 A の要素を順番に見ていき、 $exist_{A_i}$ ($1 \leq i \leq N$) を「存在している」という状態にする。

その後、整数 i ($1 \leq i \leq 200\,000 - 6$) について順番に見ていき、 $i, i+3, i+6$ が書かれているカードがすべて存在するかどうかを調べる。

すなわち、配列 $exist$ の要素を順番に見ていき、 $exist_i, exist_{i+3}, exist_{i+6}$ ($1 \leq i \leq 200\,000 - 6$) がすべて「存在している」という状態かどうかを判定する。このような i が 1 つでもあるかどうかを考えればよい。

この解法により $O(N)$ で正解が得られる。

```
1      vector<bool> exist(200000, false);
2      for(int i = 0; i < N; i++) exist[A[i]] = true;
3      for(int i = 1; i <= 200000 - 6; i++) {
4          if(flg[i] && flg[i + 3] && flg[i + 6]) {
```



第 23 回日本情報オリンピック (JOI 2023/2024) 二次予選
2023 年 12 月 10 日 (オンライン開催)

```
5         cout << "Yes" << endl;
6         return 0;
7     }
8 }
9 cout << "No" << endl;
```



2

買い物 2 (Shopping 2)

Author : 菅井 遼明

小課題 1

小課題 1 では, $N, M, Q \leq 2000$ であるから, 合計金額を求めるために, その客が買った商品を列挙しても実行時間制限内に答えることができる.

時間計算量は $O(NQ)$ となる.

```
1  #include <iostream>
2  #include <vector>
3  int main(){
4      std::ios_base::sync_with_stdio(false);
5      std::cin.tie(nullptr);
6      int N, M, Q;
7      std::cin >> N >> M >> Q;
8      std::vector<int> P(N), A(N);
9      for (int i = 0; i < N; i++){
10         std::cin >> P[i] >> A[i];
11         A[i]--;
12     }
13     for (int i = 0; i < Q; i++){
14         int T, L, R;
15         std::cin >> T >> L >> R;
16         T--;
17         L--;
18         long long ans = 0;
19         for (int j = L; j < R; j++){
20             if (A[j] == T){
21                 ans += P[j] / 2;
22             } else {
```



```
23         ans += P[j];
24     }
25 }
26 std::cout << ans << '\n';
27 }
28 }
```

小課題 2

小課題 2 では $M = 1$ である。よって、それぞれの商品の価格は必ず定価の半額となる。

商品 i ($1 \leq i \leq N$) の定価を B_i とおくと、客 k ($1 \leq k \leq Q$) が商品を買うのにかかった金額は $B_{L_k} + B_{L_k+1} + \dots + B_{R_k}$ である。よって、 B の累積和を前計算することで、それぞれの客について $O(1)$ 時間で答えを求めることができる。

時間計算量は $O(N + Q)$ となる。

```
1  #include <iostream>
2  #include <vector>
3  int main(){
4      std::ios_base::sync_with_stdio(false);
5      std::cin.tie(nullptr);
6      int N, M, Q;
7      std::cin >> N >> M >> Q;
8      std::vector<int> P(N), A(N);
9      for (int i = 0; i < N; i++){
10         std::cin >> P[i] >> A[i];
11         A[i]--;
12     }
13     std::vector<long long> S(N + 1);
14     S[0] = 0;
15     for (int i = 0; i < N; i++){
16         S[i + 1] = S[i] + P[i];
17     }
18     for (int i = 0; i < Q; i++){
19         int T, L, R;
20         std::cin >> T >> L >> R;
```



```
21     T--;  
22     L--;  
23     std::cout << (S[R] - S[L]) / 2 << '\n';  
24 }  
25 }
```

小課題 3

小課題 3 では $M \leq 10$ である。セールのそれぞれの日について、その日の各商品の価格を求めると、小課題 2 と同様に、価格の累積和を前計算することで、その日に JOI 商店を訪れた客についての答えを求めることができる。

時間計算量は $O(NM + Q)$ となる。

```
1  #include <iostream>  
2  #include <vector>  
3  int main(){  
4      std::ios_base::sync_with_stdio(false);  
5      std::cin.tie(nullptr);  
6      int N, M, Q;  
7      std::cin >> N >> M >> Q;  
8      std::vector<int> P(N), A(N);  
9      for (int i = 0; i < N; i++){  
10         std::cin >> P[i] >> A[i];  
11         A[i]--;  
12     }  
13     std::vector<std::vector<long long>> S(M, std::vector<long long>(N + 1));  
14     for (int i = 0; i < M; i++){  
15         S[i][0] = 0;  
16         for (int j = 0; j < N; j++){  
17             if (A[j] == i){  
18                 S[i][j + 1] = S[i][j] + P[j] / 2;  
19             } else {  
20                 S[i][j + 1] = S[i][j] + P[j];  
21             }  
22         }  
23     }
```



```
23     }  
24     for (int i = 0; i < Q; i++){  
25         int T, L, R;  
26         std::cin >> T >> L >> R;  
27         T--;  
28         L--;  
29         std::cout << S[T][R] - S[T][L] << '\n';  
30     }  
31 }
```

小課題 4

商品を買うのにかかった金額は、買った商品の定価の合計から、買った商品のうち価格が減ったものの価格の減少量の合計である。定価は客が JOI 商品を訪れる日によらないので、累積和を前計算することによりそれぞれの客について $O(1)$ 時間で求めることができる。

小課題 4 では、それぞれの商品の種類が異なるので、それぞれの種類の商品が高々 1 個である。よって、それぞれの客が買った商品のうち価格が減ったものは高々 1 個である。

それぞれの客 k ($1 \leq k \leq Q$) について、客 k が JOI 商品に訪れた日に価格が減少した商品があるか求め、客 k がこの商品を買ったかどうか調べることで、価格の減少量をそれぞれの客について $O(1)$ 時間で求めることができる。

時間計算量は $O(N + M + Q)$ となる。

```
1  #include <iostream>  
2  #include <vector>  
3  int main(){  
4      std::ios_base::sync_with_stdio(false);  
5      std::cin.tie(nullptr);  
6      int N, M, Q;  
7      std::cin >> N >> M >> Q;  
8      std::vector<int> P(N), A(N);  
9      for (int i = 0; i < N; i++){  
10         std::cin >> P[i] >> A[i];  
11         A[i]--;  
12     }  
13     std::vector<long long> S(N + 1);
```



```
14     S[0] = 0;
15     for (int i = 0; i < N; i++){
16         S[i + 1] = S[i] + P[i];
17     }
18     std::vector<int> p(M, -1);
19     for (int i = 0; i < N; i++){
20         p[A[i]] = i;
21     }
22     for (int i = 0; i < Q; i++){
23         int T, L, R;
24         std::cin >> T >> L >> R;
25         T--;
26         L--;
27         long long ans = S[R] - S[L];
28         if (L <= p[T] && p[T] < R){
29             ans -= P[p[T]] / 2;
30         }
31         std::cout << ans << '\n';
32     }
33 }
```

小課題 5

小課題 4 と同様に、それぞれの客について、その客が買った商品のうち価格が減ったものの価格の減少量の合計を求めることを考える。

小課題 5 では $P_i = 2$ ($1 \leq i \leq N$) であるから、半額になる商品を 1 個購入するたび、価格の減少量の合計が 1 円増加する。よって、商品 $L_k, L_k + 1, \dots, R_k$ のうち種類が T_k であるものの個数を求めればよい。

これは、以下のように解くことができる。

1. 各種類 j ($1 \leq j \leq M$) について、種類 j の商品の番号をあらかじめ昇順に列挙する。
2. 種類 T_k の商品の番号を並べた列のうち、 L_k 以上 R_k 以下である範囲を二分探索で求める。

時間計算量は $O(N + M + Q \log N)$ となる。

```
1  #include <iostream>
2  #include <vector>
```




```
3  #include <algorithm>
4  int main(){
5      std::ios_base::sync_with_stdio(false);
6      std::cin.tie(nullptr);
7      int N, M, Q;
8      std::cin >> N >> M >> Q;
9      std::vector<int> P(N), A(N);
10     for (int i = 0; i < N; i++){
11         std::cin >> P[i] >> A[i];
12         A[i]--;
13     }
14     std::vector<std::vector<int>> idx(M);
15     for (int i = 0; i < N; i++){
16         idx[A[i]].push_back(i);
17     }
18     for (int i = 0; i < Q; i++){
19         int T, L, R;
20         std::cin >> T >> L >> R;
21         T--;
22         L--;
23         int ans = (R - L) * 2;
24         ans -= std::lower_bound(idx[T].begin(), idx[T].end(), R)
25               - std::lower_bound(idx[T].begin(), idx[T].end(), L);
26         std::cout << ans << '\n';
27     }
28 }
```

小課題 6

小課題 5 の考察を発展させると、商品 $L_k, L_k + 1, \dots, R_k$ のうち種類が T_k であるものの定価の総和を求めればよい。

小課題 5 と同様に、各種類 j ($1 \leq j \leq M$) について、種類 j の商品の番号をあらかじめ昇順に列挙しておき、二分探索を行うことで、種類 T_k の商品のうち何番目から何番目まで客 k が買ったかを求めることができる。種類 T_k の商品に限定した場合でのこの範囲の定価の総和を求めればよい。各種類 j ($1 \leq j \leq M$) に



ついて種類 j の商品に限定して定価の累積和を計算することで、定価の総和も高速に求めることができる。
時間計算量は $O(N + M + Q \log N)$ となる。

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  int main(){
5      std::ios_base::sync_with_stdio(false);
6      std::cin.tie(nullptr);
7      int N, M, Q;
8      std::cin >> N >> M >> Q;
9      std::vector<int> P(N), A(N);
10     for (int i = 0; i < N; i++){
11         std::cin >> P[i] >> A[i];
12         A[i]--;
13     }
14     std::vector<long long> S(N + 1);
15     S[0] = 0;
16     for (int i = 0; i < N; i++){
17         S[i + 1] = S[i] + P[i];
18     }
19     std::vector<std::vector<int>> idx(M);
20     for (int i = 0; i < N; i++){
21         idx[A[i]].push_back(i);
22     }
23     std::vector<std::vector<long long>> sum(M);
24     for (int i = 0; i < M; i++){
25         int cnt = idx[i].size();
26         sum[i].resize(cnt + 1);
27         sum[i][0] = 0;
28         for (int j = 0; j < cnt; j++){
29             sum[i][j + 1] = sum[i][j] + P[idx[i][j]];
30         }
31     }
32     for (int i = 0; i < Q; i++){
```



```
33     int T, L, R;
34     std::cin >> T >> L >> R;
35     T--;
36     L--;
37     long long ans = S[R] - S[L];
38     int l = std::lower_bound(idx[T].begin(), idx[T].end(), L) - idx[T].begin();
39     int r = std::lower_bound(idx[T].begin(), idx[T].end(), R) - idx[T].begin();
40     ans -= (sum[T][r] - sum[T][l]) / 2;
41     std::cout << ans << '\n';
42 }
43 }
```



3

白色光 2 (White Light 2)

Author : 蜂矢 倫久 (Mitsubachi)

小課題 1

点灯しているライトの数は 3 の倍数である。したがって、 $N = 3$ のときには点灯しているライトの数は 0 または 3 である。

点灯しているライトの数を 0 にするために必要な金額の最小値は $A \leq B$ のとき $3A$ で、そうでないときは $3B$ である。点灯しているライトの数を 3 にするとき、ライトの色の並びは 'RGB' となる。したがって、 S の i 文字目と 'RGB' の i 文字目が異なるような i の数に C をかけた値が必要な金額の最小値である。

答えは上で求めた 2 つの値のうち小さい方である。 $O(N)$ で答えが求まる。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  string rgb = "RGB";
5  int main(){
6      int n;
7      string s;
8      long long a, b, c;
9      cin >> n >> s >> a >> b >> c;
10
11     long long ans = 0;
12     for(int i = 0; i < 3; i++){
13         if(s[i] != rgb[i]){
14             ans += c;
15         }
16     }
17     ans = min(ans, min(a, b) * 3);
18     cout << ans << endl;
19 }
```



小課題 2

ライトをすべて消灯させるのに必要な金額の最小値は小課題 1 のようにして求められる。いくつかのライトが点灯している状態で操作が終わる場合を考える。

$1 \leq L \leq R \leq N$, $R - L + 1$ を 3 の倍数として、操作を終えたときに点灯しているようなライトの範囲を左から L 番目から R 番目までと固定する。このとき、ライトを消灯させるのに必要なコストは $(L - 1)A + (N - R)B$ である。

また L 番目から R 番目のライトのうち赤色へ点灯し直す必要があるものの個数は $L \leq i \leq R$, $i - L$ を 3 で割ったあまりが 0 となる i の中でライト i が赤色でないものの個数である。同様に、緑色へ点灯し直す必要があるものと青色へ点灯し直す必要があるものの個数も分かる。

これより、操作を終えたときに点灯しているようなライトの範囲を左から L 番目から R 番目までと固定したときの必要な金額の最小値は $O(N)$ で求められる。点灯しているライトの範囲として考えられるものは $O(N^2)$ 通りあるので全体で $O(N^3)$ で答えが求まる。

```
1      long long ans = 1e18;
2
3      for(int i = 0; i <= n; i++){
4          for(int j = i; j <= n; j++){
5              // ライト i からライト j-1 まで点灯させる ただし i = j のときはすべて消灯する
6              if((j - i) % 3 != 0){
7                  continue;
8              }
9              long long cost_now = a * i + b * (n - j);
10             for(int k = i; k < j; k++){
11                 if(s[k] != rgb[(k - i) % 3]){
12                     cost_now += c;
13                 }
14             }
15             ans = min(ans, cost_now);
16         }
17     }
```



小課題 3

$1 \leq L \leq R \leq N-3$, $R-L+1$ を 3 の倍数として, 操作を終えたときに点灯しているようなライトの範囲を左から L 番目から R 番目までと固定したときの, 必要な金額の最小値が分かっているとする. このとき, 操作を終えたときに点灯しているようなライトの範囲を左から L 番目から $R+3$ 番目までと固定したときの, 必要な金額の最小値を求める.

ライトを消灯させるのに必要なコストは $3B$ 減少する. また L 番目から $R+3$ 番目のライトのうち点灯し直す必要があるものの個数は, L 番目から R 番目のライトのうち点灯し直す必要があるものの個数に $R+1$ 番目から $R+3$ 番目までの中で点灯し直す必要があるものの個数を足したものである. したがって, $O(1)$ で左から L 番目から $R+3$ 番目までと固定したときの, 必要な金額の最小値も分かる.

よって操作を終えたときに点灯しているようなライトの範囲を L の小さい順に, L が同じときは R が小さい順に固定したときの必要な金額の最小値を求めることで全体で $O(N^2)$ で答えが求まる.

```
1      long long ans = 1e18;
2
3      for(int i = 0; i <= n; i++){
4          long long cost_now = a * i;
5          for(int j = i; j < n; j++){
6              // ライト i からライト j までを点灯させる
7              // cost_now: B 円を払う操作以外の操作に必要な金額
8              if(s[j] != rgb[(j - i) % 3]){
9                  cost_now += c;
10             }
11             if((j - i + 1) % 3 == 0){
12                 ans = min(ans, cost_now + b * (n - 1 - j));
13             }
14         }
15     }
16
17     // ライトをすべて消灯させる場合
18     ans = min(ans, min(a, b) * n);
```



小課題 3 別解

$1 \leq L \leq R \leq N$, $R - L + 1$ を 3 の倍数として, 操作を終えたときに点灯しているようなライトの範囲を左から L 番目から R 番目までと固定する. このとき, ライトを消灯させるのに必要なコストは $(L - 1)A + (N - R)B$ である.

また L 番目から R 番目のライトのうち操作の前後で共に赤色となる個数は $L \leq i \leq R$, $i - L$ を 3 で割ったあまりが 0 となる i の中でライト i が赤色であるものの個数である. 同様にして, 緑色と青色の場合の個数も分かる. これらの個数は前もって累積和を取ることで $O(1)$ で入手できる.

これより, 操作を終えたときに点灯しているようなライトの範囲を左から L 番目から R 番目までと固定したときの必要な金額の最小値は $O(1)$ で求められる. 全体で $O(N^2)$ で答えが求まる.

```
1 // 累積和を取る
2 vector<vector<int>> sum(n, vector<int>(3, 0));
3 for(int i = 0; i < n; i++){
4     for(int j = 0; j < 3; j++){
5         if(s[i] == rgb[j]){
6             sum[i][j]++;
7         }
8         if(i > 2){
9             sum[i][j] += sum[i - 3][j];
10        }
11    }
12 }
13
14 long long ans = min(a, b) * n;
15 for(int i = 0; i < n; i++){
16     for(int j = i; j < n; j++){
17         if((j - i + 1) % 3 != 0){
18             continue;
19         }
20         // 何個点灯し直すか
21         int change = j - i + 1;
22
23         // 最後赤色になっていて, かつ最初から赤色になっている数を引く
```



```
24         change -= sum[i + (j - i) / 3 * 3][0];
25         if(i > 2){
26             change += sum[i - 3][0];
27         }
28
29         // 緑色や青色についても同様
30         change -= sum[i + 1 + (j - i - 1) / 3 * 3][1];
31         if(i > 1){
32             change += sum[i - 2][1];
33         }
34
35         change -= sum[i + 2 + (j - i - 2) / 3 * 3][2];
36         if(i > 0){
37             change += sum[i - 1][2];
38         }
39
40         ans = min(ans, a * i + b * (n - 1 - j) + c * change);
41     }
42 }
```

小課題 4

消灯させる操作を 1 回でも行くと 10^9 円が少なくとも必要だが、すべてのライトを点灯し直す操作のみで条件を満たすようにしても高々 N 円で操作は完了する。よって、ライトを点灯し直す操作のみを考えればよい。

小課題 2 で述べたように、赤色へ点灯し直す必要があるものの個数は i を 3 で割ったあまりが 0 となる i の中でライト i が赤色でないものの個数である。同様に、緑色へ点灯し直す必要があるものと青色へ点灯し直す必要があるものの個数も分かる。

これより、点灯し直す必要のあるライトの個数が $O(N)$ で分かるので、全体で $O(N)$ で答えが求まる。



```
1      long long ans = 0;
2      for(int i = 0; i < n; i++){
3          if(s[i] != rgb[i % 3]){
4              ans += c;
5          }
6      }
```

小課題 5

N を 3 で割ったあまりを x として、条件を満たすとき点灯しているライトの数は 3 の倍数なので x 回は消灯させる操作を行う必要がある。その後ライトを点灯し直す操作を行うとして条件を満たすためには高々 $N - x$ 回で十分である。よって必要な金額の最小値は $x \times 10^9 + N - x$ である。逆に、消灯させる操作を x 回より多く行う場合の必要な金額の最小値は少なくとも $(x + 3) \times 10^9$ である。

これより消灯させる操作の回数は x 回としてよい。よって操作を終えたときに点灯しているようなライトの範囲は $x + 1$ 通りのみを考えればよく、全体で $O(N)$ で答えが求まる。実装の際、左から消灯する個数と右から消灯する個数を 0 から 2 個まで全探索しても十分高速である。

```
1      long long ans = 1e18;
2
3      for(int i = 0; i < 3; i++){
4          for(int j = 0; j < 3; j++){
5              // 先頭から i 個のライト, 末尾から j 個のライトを消灯させた場合
6              if((n - i - j) % 3 != 0 || n < i + j){
7                  continue;
8              }
9              int left = i, right = n - j;
10             long long cost_now = i * a + j * b;
11             for(int k = left; k < right; k++){
12                 if(s[k] != rgb[(k - left) % 3]){
13                     cost_now += c;
14                 }
15             }
16             ans = min(ans, cost_now);
17         }
18     }
```



満点

操作を終えるときに点灯しているライトの範囲が R 番目までとして、 R の小さい順に必要な金額の最小値を求める。右から消灯させる操作に必要な金額は $(N - R)B$ である。よって、1 番目から R 番目までの操作について必要な金額の最小値を考えればよい。

$R > 3$ のとき、1 番目から $R - 3$ 番目までの操作について必要な金額の最小値を x とする。 $R - 2$ 番目から R 番目を点灯させる場合、 $R - 2$ 番目から R 番目で点灯し直すライトの数を y として必要な金額の最小値は $x + yC$ である。 R 番目まで左から消灯させるとき、必要な金額は RA である。

よって、1 番目から R 番目までの操作について必要な金額の最小値はこの 2 つの値のうち小さい方である。これより、点灯しているライトの範囲が R 番目までとして、1 番目から R 番目までの操作について必要な金額の最小値は R の小さい順に $O(N)$ ですべて求まる。

全体で $O(N)$ で答えが求まる。

```
1      long long ans = min(a, b) * n;
2
3      // i 番目までの操作について必要な金額の最小値を opt[i % 3] に保持し、更新していく
4      vector<long long> opt = {a, 2 * a, 0};
5
6      for(int i = 2; i < n; i++){
7          if(s[i - 2] != 'R'){
8              opt[i % 3] += c;
9          }
10         if(s[i - 1] != 'G'){
11             opt[i % 3] += c;
12         }
13         if(s[i] != 'B'){
14             opt[i % 3] += c;
15         }
16
17         opt[i % 3] = min(opt[i % 3], (i + 1) * a);
18         ans = min(ans, opt[i % 3] + (n - 1 - i) * b);
19     }
```



別解

各ライトについて操作を終了したときの状態は次のいずれかである。

- 状態 1: 赤色に点灯している。
- 状態 2: 緑色に点灯している。
- 状態 3: 青色に点灯している。
- 状態 4: A 円を払う操作により消灯している。
- 状態 5: B 円を払う操作により消灯している。

ここで、条件を満たすとき、ライトの状態について以下のように考えられる。

- 状態 1 のライトについて、そのライトは左端または左隣のライトは状態 3 か状態 4 である。
- 状態 2 のライトについて、左隣のライトは状態 1 である。
- 状態 3 のライトについて、左隣のライトは状態 2 である。
- 状態 4 のライトについて、そのライトは左端または左隣のライトは状態 4 である。
- 状態 5 のライトについて、そのライトは左端または左隣のライトは状態 3 か状態 5 である。

よって、ライト i が状態 j である条件を満たすライトの状態において、ライト 1 からライト i までに
関する操作に必要な金額の最小値を求める動的計画法を考えることでこの問題は $O(N)$ で解くことができる。

```
1 // dp[i][j]: ライト i が状態 j であり条件を満たしているときの金額の最小値
2 vector<vector<long long>> dp(n + 1, vector<long long>(5, 1e18));
3 dp[0][3] = 0;
4
5 for(int i = 0; i < n; i++){
6     for(int j = 0; j < 3; j++){
7         int k = (j + 2) % 3;
8
9         if(s[i] != rgb[j]){
10             dp[i + 1][j] = min(dp[i + 1][j], dp[i][k] + c);
11             if(j == 0){
12                 dp[i + 1][j] = min(dp[i + 1][j], dp[i][3] + c);
13             }
14         }
15         else{
```



```
16         dp[i + 1][j] = min(dp[i + 1][j], dp[i][k]);
17         if(j == 0){
18             dp[i + 1][j] = min(dp[i + 1][j], dp[i][3]);
19         }
20     }
21 }
22
23 dp[i + 1][3] = min(dp[i + 1][3], dp[i][3] + a);
24
25 for(int j = 2; j < 5; j++){
26     dp[i + 1][4] = min(dp[i + 1][4], dp[i][j] + b);
27 }
28 }
29
30 long long ans = 1e18;
31 for(int j = 2; j < 5; j++){
32     ans = min(ans, dp[n][j]);
33 }
```



4

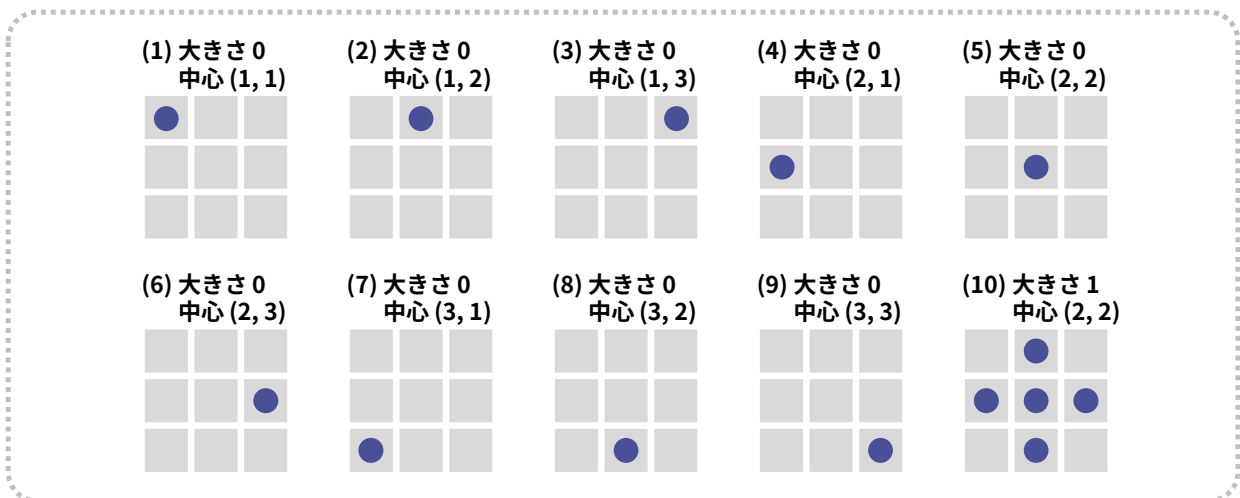
庭園 2 (Garden 2)

解説作成者：米田優峻

小課題 1 (累計 4 点)

$N = 3$ の場合、花の植え方としては以下の 10 通りがあり、そのうち (1)~(9) までの 9 通りはどんな入力の場合でも選択することができます*¹。しかし、(10) は区画 (1, 2), 区画 (2, 1), 区画 (2, 3), 区画 (3, 2) の色がすべて同じでなければ選択できません。

したがって、 $A_{1,2} = A_{2,1} = A_{2,3} = A_{3,2}$ のとき答えは 5 本、そうでないとき答えは 1 本となります。if 文などの条件分岐を使うと簡単に実装することができます。



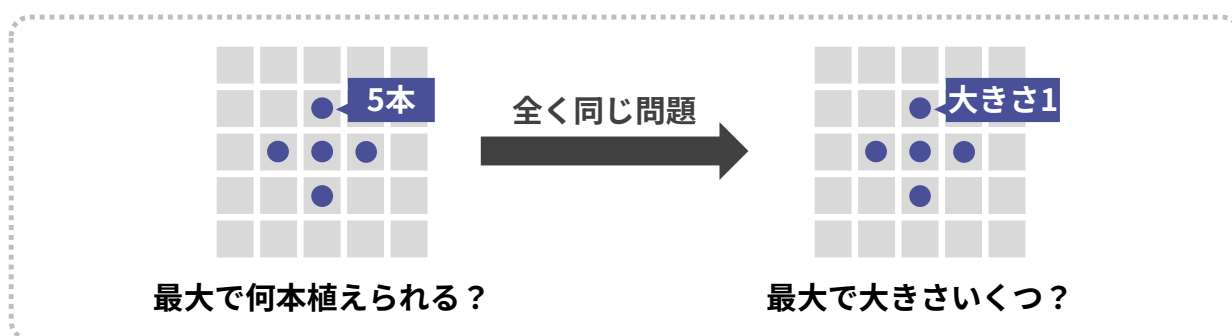
(次ページへ続く)

*¹ たとえばパターン (1) の場合、色 $A_{1,1}$ の花を区画 (1, 1) に植えればよいです。



小課題 2 の前に

まず、本問題では最大何本の花を植えられるかを出力しなければなりません。しかし、大きさ r が決まってしまうと、植える花の数は $2r^2 + 2r + 1$ 本と自動的に決まります。そのため、最大で大きさいくつにできるかが分かれば、答えも分かります。そのため以降の解説では、大きさの最大値を求めることを考えます。

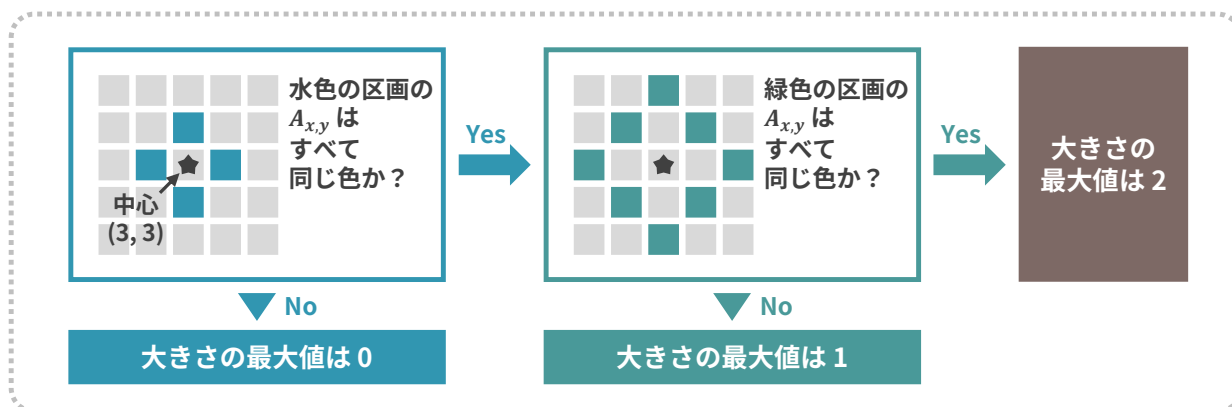


小課題 2 (累計 17 点)

この小課題は、中心の区画 (x, y) を全探索して、それぞれの区画に対して「達成できる大きさ r の最大値 $r_{\max}$ 」を計算することで解けます。

しかし、達成できる大きさの最大値 $r_{\max}$ はどうやって計算すれば良いのでしょうか。例として、庭園の大きさが 5×5 であり、中心が区画 $(3, 3)$ の場合を考えてみましょう。以下ようになります。

- $|x' - 3| + |y' - 3| = 1$ を満たす区画 (x', y') について、色 $A_{x', y'}$ が同じでなければ、 $r_{\max} = 0$
- $|x' - 3| + |y' - 3| = 2$ を満たす区画 (x', y') について、色 $A_{x', y'}$ が同じでなければ、 $r_{\max} = 1$
- そうでなければ、 $r_{\max} = 2$





中心が区画 (3, 3) ではない場合も、同じような方法で $r_{\max}$ を計算することができます。具体的には、中心が (x, y) のときの $|x' - x| + |y' - y|$ の値を 距離 と呼ぶとき、以下ようになります。

- 距離が 1 の区画 (x', y') について、色 $A_{x', y'}$ が同じでなければ、 $r_{\max} = 0$
- 距離が 2 の区画 (x', y') について、色 $A_{x', y'}$ が同じでなければ、 $r_{\max} = 1$
- 距離が 3 の区画 (x', y') について、色 $A_{x', y'}$ が同じでなければ、 $r_{\max} = 2$
- 以下同様

これを自然に実装すると $r_{\max}$ を求めるのに計算量 $O(N^2)$ を要し、中心のパターン数が $O(N^2)$ 通りであるため、全体の計算量が $O(N^4)$ になりますが、 $N \leq 50$ という制約下では十分高速に動作します。

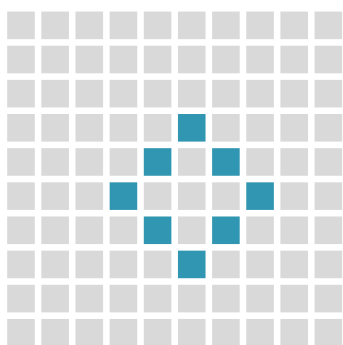
小課題 3 (累計 34 点)

小課題 2 では、 $r_{\max}$ を計算するのに $O(N^2)$ を要しました。しかし、この計算を $O(N)$ にしなければ、小課題 3 の $N \leq 800$ という制約下で通すことはできません。一体どうすれば良いのでしょうか。

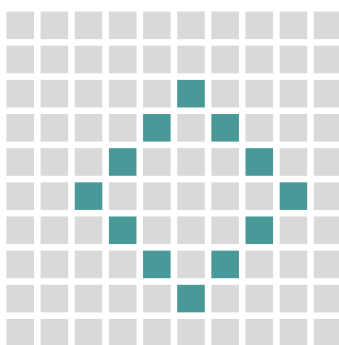
$r_{\max}$ の値を計算するには、中心からの距離が i である区画の色がすべて同じかどうかを、 $i = 1, 2, 3, \dots$ について判定する必要がありました。つまり下図のような区画の色がすべて同じかどうかを、 $i = 1, 2, 3, \dots$ について判定する必要がありました。

ここで中心からの距離が i である区画は最大 $O(N)$ 個存在するため、愚直に実装すると各 i について計算量 $O(N)$ 、全体で $O(N^2)$ かかります。

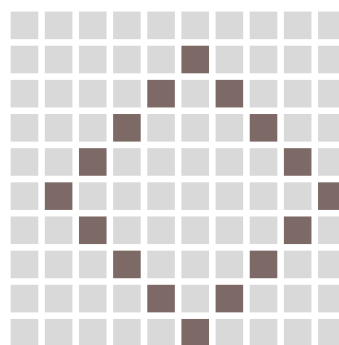
(1) $i = 2$ の例



(2) $i = 3$ の例

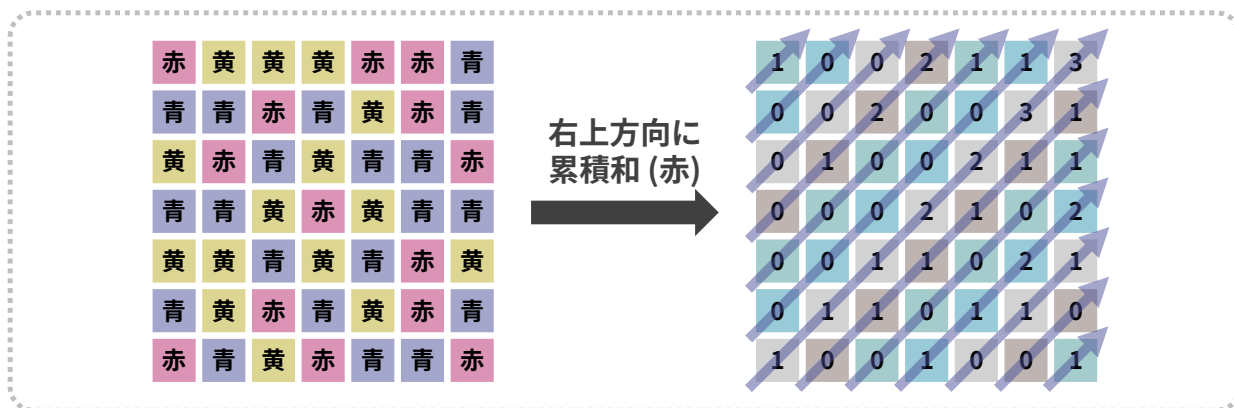


(3) $i = 4$ の例



(次ページへ続く)

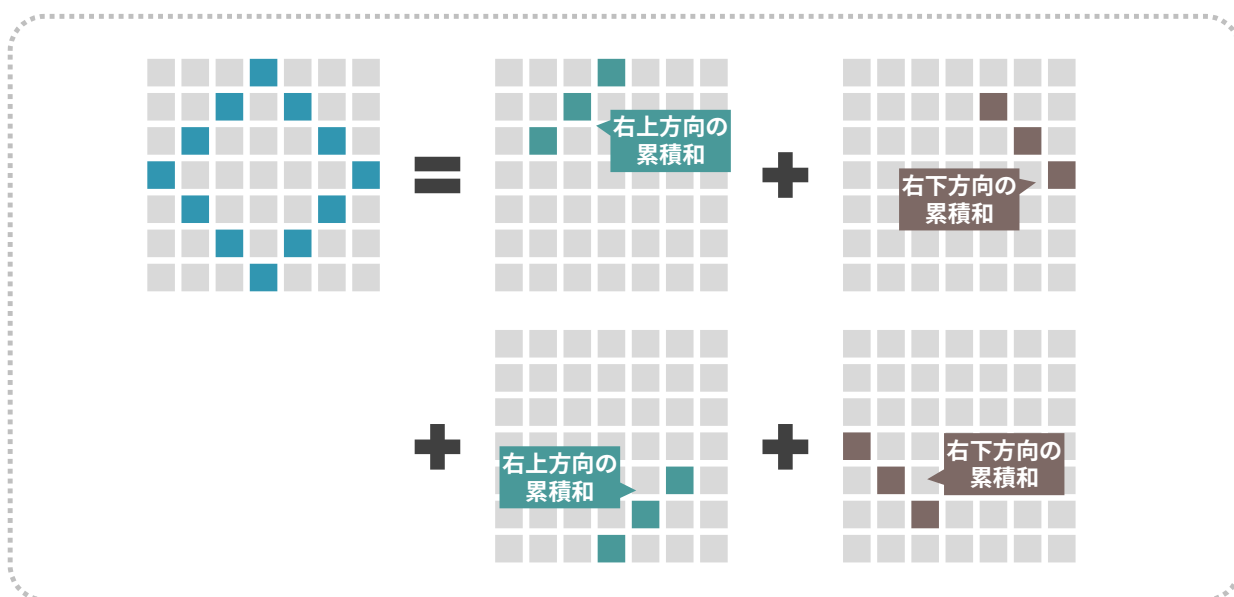
しかし、累積和を使うと各 i に対する計算量が $O(1)$ になります。まず、赤/黄/青それぞれについて、 $A_{x,y}$ がその色である個数を右上方向に累積和を取ります。赤の場合の例を下図に示します。(たとえば上から 4 行目の斜めは左から順に青・赤・赤・黄となっているため、累積和は 0, 1, 2, 2 となります)



次に、同じような累積和を右下方向にも取ります。すると、下図のようにして次の値を計算量 $O(1)$ で求めることができるため、各 i に対して「色がすべて同じかどうか」を $O(1)$ で判定できます。

- 中心からの距離が i である区画の中で、 $A_{x,y}$ が赤であるような区画の個数
- 中心からの距離が i である区画の中で、 $A_{x,y}$ が黄であるような区画の個数
- 中心からの距離が i である区画の中で、 $A_{x,y}$ が青であるような区画の個数

したがって、 $r_{\max}$ の計算にかかる時間が $O(N)$ に減り、全体で計算量 $O(N^3)$ となって小課題 3 に通ります。



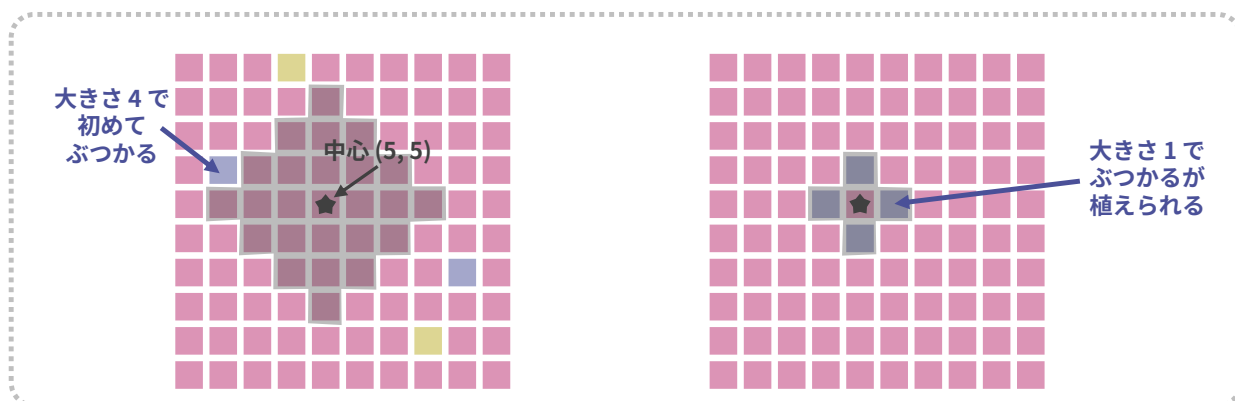


小課題 4 (累計 48 点)

この小課題では、 $A_{x,y}$ が赤以外の色であるような区画が高々 5 個しか存在しません。そのため、中心 (x,y) を決めて大きさ r をだんだん増やしていったとき、基本的には初めて赤以外の区画とぶつかったタイミングで植えられなくなります。たとえば下図左側の場合、大きさが 4 のときに初めて赤以外の区画とぶつかるため、大きさ 3 までしか植えることができません。

したがって、 $r_{\max}$ の値は中心から最も近い「赤以外の区画」までの距離に 1 を引いた値となります。この性質を使うと、愚直に実装しても $5N^2$ 回程度の計算で答えが出るため、小課題 4 に通ります。

ただし、大きさ 1 の場合は下図右側のように、赤以外の区画とぶつかっても植えられるケースが存在します。場合分けをする必要があることに注意してください。



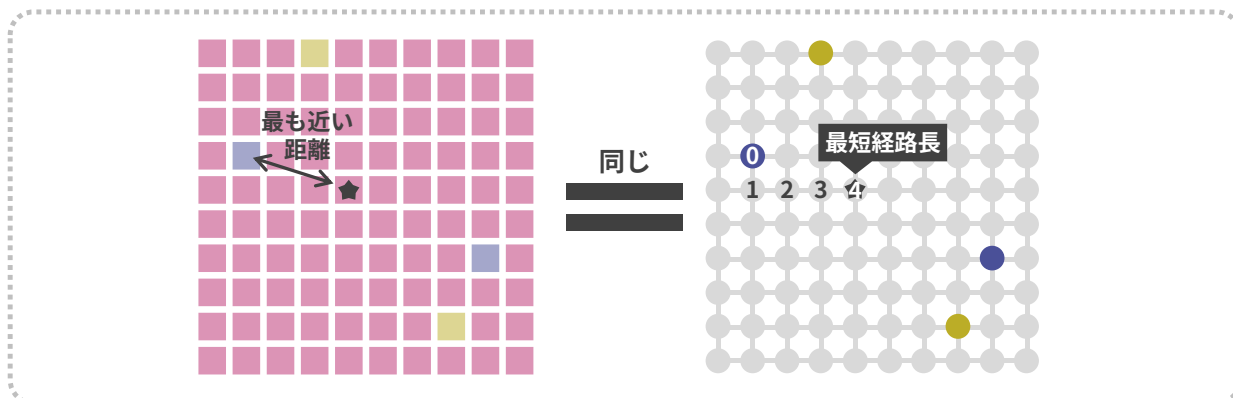
小課題 5 (累計 64 点)

この小課題では、どの 2×2 の範囲にも赤の区画が 3 つ以上存在します。そのため、小課題 4 と同様、 $r_{\max}$ の値は「中心から最も近い赤以外の区画までの距離^{*2}」に 1 を引いた値となります。

しかし小課題 4 とは違って、 $A_{x,y}$ が赤以外の色であるような区画がたくさん (具体的には $N^2/4$ 個程度) あるため、自然に実装すると TLE になってしまいます。

そこで各区画を頂点、上下左右に隣り合う区画を辺とした重み無しグラフを考えます。すると、「中心から最も近い赤以外の区画までの距離」は、「グラフで赤以外の頂点から出発した時の最短経路の長さ」と同じなので、最短経路長ささえ分かれば答えが求まります。(次ページの図を参照)

^{*2} ただし、自分自身の区画は含めないものとします。

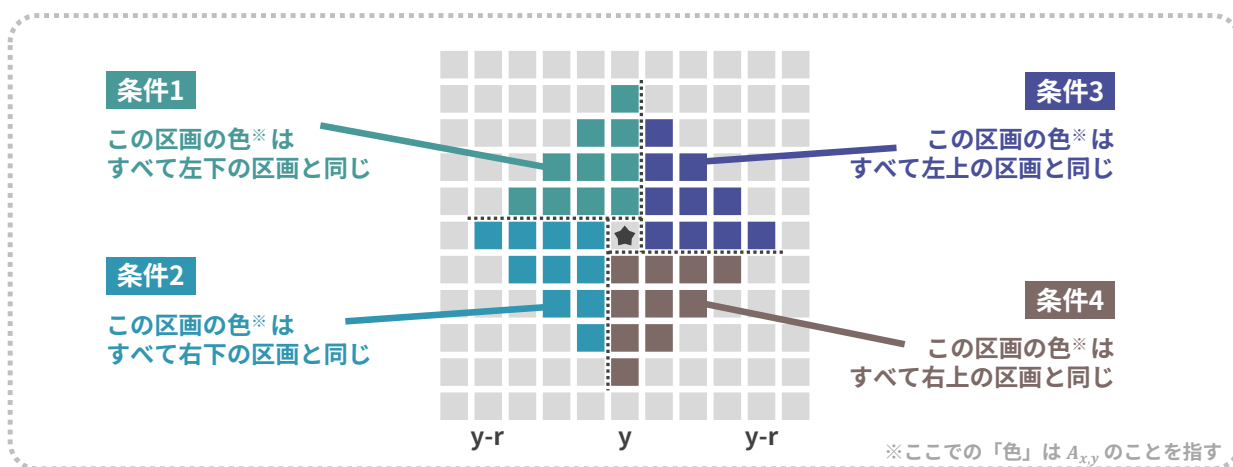


それでは、最短経路長はどうやって求めれば良いのでしょうか。出発地点が 1 つの場合は、まず出発地点をキューに入れてから幅優先探索をすれば良いですが、今回は出発地点 (赤以外の区画) がたくさんあるので、何か工夫が必要だと思うかもしれません。

しかし、出発地点が 1 つの場合と同様、単純にすべての出発地点をキューに入れてから幅優先探索をしても、計算量 $O(N^2)$ で各頂点への最短経路長が求まります^{*3}。なお、本小課題では中心以外に青や黄の花を植えることはありませんが、中心が青や黄になるケースもあるので、場合分けに注意してください。

小課題 6 (満点)

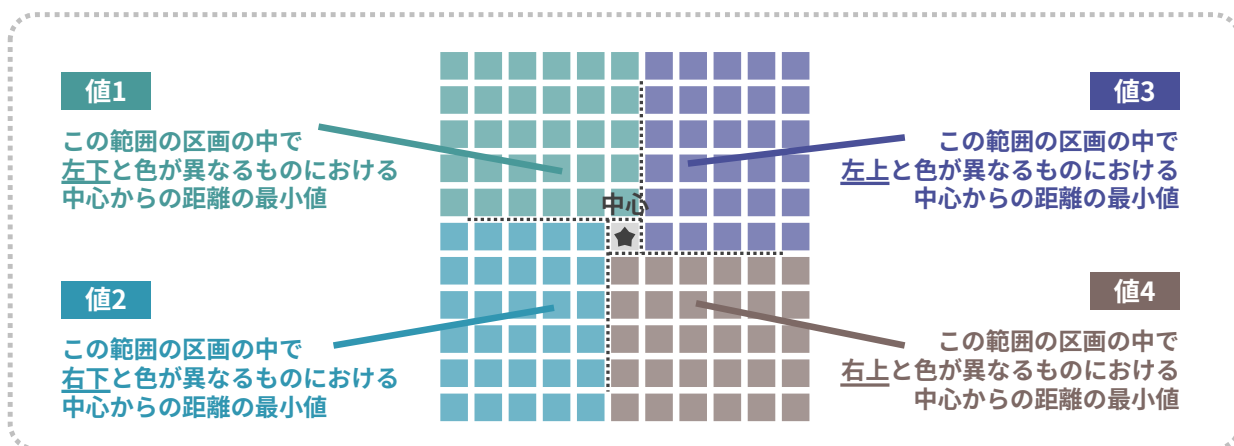
まず、中心を区画 (x,y) と決めたときに、大きさ r を達成できるための必要十分条件は、以下の図に書かれている 4 つの条件をすべて満たすことです。



^{*3} このようになる理由は、もし出発地点が複数でも、各頂点がキューに 1 回ずつしか入らないからです。



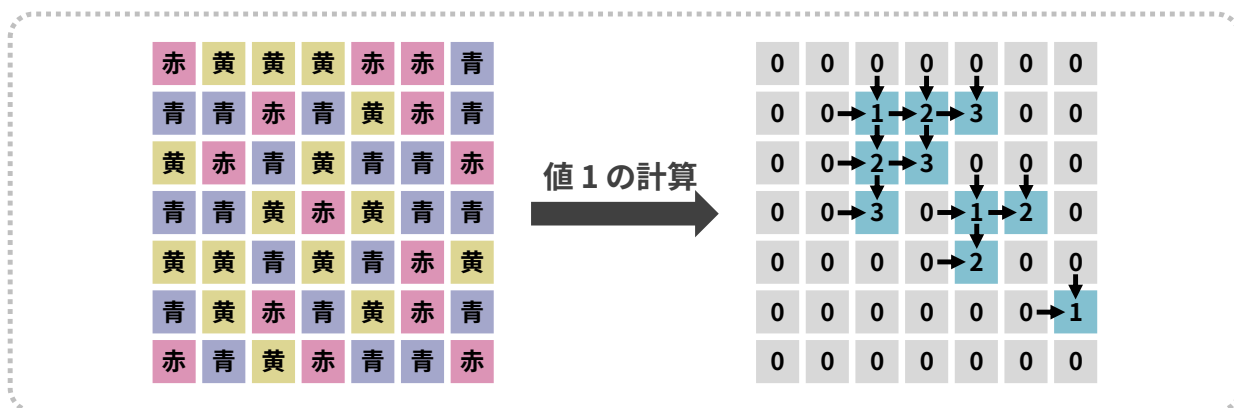
したがって、中心を区画 (x, y) と決めたときに達成できる最大の大きさ $r_{\max}$ は、下図の値 1, 値 2, 値 3, 値 4, および $\max(x-1, y-1, N-x, N-y)^{*4}$ の最小値となります。



それでは、値 1, 値 2, 値 3, 値 4 はどうやって求めれば良いのでしょうか。実は動的計画法を使えば簡単です。たとえば値 1 の場合、以下のような式になります。

- $dp[0][j] = 0$
- $dp[i][0] = 0$
- $A_{i,j} = A_{i+1,j-1}$ の場合, $dp[i][j] = \min(dp[i-1][j], dp[i][j-1]) + 1$
- $A_{i,j} \neq A_{i+1,j-1}$ の場合, $dp[i][j] = 0$
- 中心が (x, y) のときの値 1 は $dp[x-1][y]$ に相当

動的計画法の計算の例を下図に示します。(右の図では, $A_{i,j} = A_{i+1,j-1}$ の区画を濃い色で示しています)



^{*4} x, y は $r+1 \leq x \leq N-r$, $r+1 \leq y \leq N-r$ を満たすように選ぶ必要があるため、たとえば全部 $A_{x,y}$ が同じであるようなケースでも、 $r \leq \max(x-1, y-1, N-x, N-y)$ を満たす必要があることに注意してください。



第 23 回日本情報オリンピック (JOI 2023/2024) 二次予選
2023 年 12 月 10 日 (オンライン開催)

以上のことを行くと，値 1，値 2，値 3，値 4 の計算を $O(N^2)$ 時間，各中心に対する $r_{\max}$ の値の計算を $O(1)$ 時間で行うことができるため，全体で計算量 $O(N^2)$ で本問題を解くことができました。

なお，この問題は実装がやや複雑ですが，情報オリンピックでは過去にも複雑な実装の問題が何問か出題されているので，来年以降チャンスのある選手はぜひ実装力もつけておきましょう。

(解説は以上です)



5

高速道路の通行料金 (Highway Tolls)

Author : 渡邊 雄斗

共通の考察

通行料金の総和の最小値を求める上では、移動方法について以下の制約を加えても問題ありません（証明は省略しますが、後述の「小課題 4 以降に共通の考察」から自然と導かれます）。

- 高速道路を通行せずにどこかの都市に留まっている時間はないものとする。
- 都市 1 を出発する時刻は整数時刻とする。

なお、 L_i が整数であることから、以上の 2 つの制約が満たされるとき、任意の都市を出発する時刻が整数時刻になることに注意してください。

小課題 1

通行料金は時刻によらず一定なので、単純な最短経路問題になります。最短経路問題を解くアルゴリズムとしては Dijkstra 法, Bellman-Ford 法, Floyd-Warshall 法などがよく知られていますが、そのいずれを用いても解くことができます。

小課題 2

以下のような動的計画法を考えます。

$dp[j][t]$: 時刻 t に都市 j にいるとき、ここから都市 N まで移動するのに必要な通行料金の総和の最小値

遷移としては都市 j から伸びている各高速道路の通行について考えればよく、現在の時刻 t を情報として持っているため、その通行料金は簡単に計算できます。

また、時刻 t としてありうる値は本来無限にありますが、正の時刻に都市 1 を出発するような経路や負の時刻に都市 N に到着するような経路を考える必要はない（*）ため、 t の絶対値は高々 $N \times \max L_i$ 程度としてしまってもよいです。よって、計算量は $O(N(N + M) \times \max L_i)$ です。



(*) の理由：正の時刻に都市 1 を出発する場合，代わりに時刻 0 に都市 1 を出発して全く同じ経路を辿っても通行料金の総和は増加しないため．負の時刻に都市 N に到着する場合についても同様．

小課題 3

経路は 1 通りに定まっているので，都市 1 を出発する時刻についてのみ考えればよいです．

都市 1 を出発してから都市 j に到着するまでの時間を $S_j (= L_1 + L_2 + \dots + L_{j-1})$ とおきます．都市 1 を出発する時刻を t とすると，通行料金の総和は $\sum_{i=1}^{N-1} (C_i + K \times |t + S_i|) = K \sum_{i=1}^{N-1} |t + S_i| + \sum_{i=1}^{N-1} C_i$ となるため， $\sum_{i=1}^{N-1} |t + S_i|$ を最小化すればよいです． $S_1 < S_2 < \dots < S_{N-1}$ に注意すると，この値は $t = -S_{\lfloor \frac{N}{2} \rfloor}$ のときに最小値をとる (**) ため， $O(N)$ で答えを求めることができます．

(**) の理由： $f(t) = \sum_{i=1}^{N-1} |t + S_i|$ とおき， $t' = S_{\lfloor \frac{N}{2} \rfloor}$ とします．このとき， $f(t)$ は $t \leq t'$ の範囲で単調減少し $t \geq t'$ の範囲で (広義) 単調増加します (証明略)．よって， $f(t)$ は $t = t'$ で最小値をとります．
なお，これは「数の集合 $A = \{A_1, A_2, \dots, A_k\}$ が与えられたとき，関数 $f(x) = |x - A_1| + |x - A_2| + \dots + |x - A_k|$ は x が A の中央値のとき最小値をとる」という事実としてよく知られているものです．

小課題 4 以降に共通の考察

小課題 3 の考察を用いると，経路を 1 つ決め打ったときの通行料金の総和の最小値を各道路の寄与に分解した形で表すことができます．例えば，道路 w, x, y, z をこの順に通行して都市 1 から都市 N まで移動するとき，前述の考察より時刻 $-L_w$ に都市 1 を出発するのが最適ですから，通行料金の総和の最小値は $(C_w + K \times |-L_w|) + (C_x + K \times 0) + (C_y + K \times |L_x|) + (C_z + K \times |L_x + L_y|) = (C_x + KL_w) + (C_x + 2KL_x) + (C_y + KL_y) + (C_z)$ となります．より一般に，通行料金の総和の最小値は通行する全ての道路に対する $C_i + \text{coef}_i \times KL_i$ の総和になります．ここで，係数 coef_i は「通行する道路の数」と「その道路を何番目に通行するか」から定まる係数です．

小課題 4

通行する道路の数，およびそれぞれの道路についてその道路を通行する場合何番目に通行するかが一通りに定まります．よって，上述した各道路の寄与 $C_i + \text{coef}_i \times KL_i$ は経路に依存しないので，単純な最短経路問題になります．



小課題 5

通行する道路の数を固定してみます。このとき、各道路の寄与は「その道路を何番目に通行するか」にのみ依存しますから、今いる都市の情報に加えて今までに通行した道路の数の情報を付与した $O(N^2)$ 頂点の拡張グラフ上では各道路の寄与が一意に定まり、単純な最短経路問題に帰着されます。通行する道路の数を全探索し、帰着された最短経路問題を解くのに dijkstra 法を用いることで、 $O(N^2(N+M)\log N)$ の計算量で解くことができます。

小課題 6

各経路の寄与を定める係数 coef_i についてもう少し考察を進めてみましょう。

何らかの経路を定めたとき、前述の考察より、最適解においては時刻 0 ぴったりに訪れるような都市が必ず存在します。この都市を仮に a とおくと、以下の事実が観察できます。

- 都市 1 から都市 a に至るまでに通行する道路を通行順に並べたとき、それらの道路の係数は $1, 2, 3, \dots$ というようになっている。
- 都市 a から都市 N に至るまでに通行する道路を通行順と逆順に並べたとき、それらの道路の係数は $0, 1, 2, \dots$ というようになっている。

時刻 0 ぴったりに訪れるような都市 a を 1 つ固定し、経路を「都市 1 から都市 a まで移動するパート」と「都市 N から都市 a まで高速道路を逆走して移動するパート」に分けて考えます。すると、上述の考察より、

- 各 $1 \leq v \leq N$, $1 \leq p \leq N$ について、都市 1 から都市 v まで p 本の道路を通行して移動するのに必要な通行料金の総和の最小値
- 各 $1 \leq v \leq N$, $1 \leq p \leq N$ について、都市 N から都市 v まで p 本の道路を逆走して移動するのに必要な通行料金の総和の最小値

が小課題 5 と同様の dijkstra 法によって $O(N(N+M)\log N)$ で求まるので、それぞれのパートにおける必要な通行料金の総和の最小値が計算できます。これらの情報は a を変えるたびに再計算する必要はなく、最初に 1 度求めてしまえばよいものですから、 a を全探索しても変わらず $O(N(N+M)\log N)$ の計算量で解くことができます。

満点

「都市 1 から都市 v まで p 本の道路を通行して移動するのに必要な通行料金の総和の最小値」を $\text{dp}[v][p]$ とおきましょう。この DP テーブルの各値は p の昇順に見ていけば単純な計算で求めることができ、最短経



第 23 回日本情報オリンピック (JOI 2023/2024) 二次予選
2023 年 12 月 10 日 (オンライン開催)

路問題に帰着して `dijkstra` 法を用いる必要は実はありません。「都市 N から都市 v まで p 本の道路を逆走して移動するのに必要な通行料金の総和の最小値」についても同様ですから、計算量は $O(N(N + M))$ になり、満点を獲得できます。